



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



TFG del Grado en Ingeniería
Informática

título del TFG
Documentación Técnica



Presentado por nombre alumno
en Universidad de Burgos — 25 de febrero
de 2026

Tutor: nombre tutor

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	iv
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	2
A.3. Estudio de viabilidad	11
Apéndice B Especificación de Requisitos	13
B.1. Introducción	13
B.2. Objetivos generales	13
B.3. Catálogo de requisitos	13
B.4. Especificación de requisitos	13
B.5. Actores del sistema	15
Apéndice C Especificación de diseño	25
C.1. Introducción	25
C.2. Diseño de datos	25
C.3. Diseño arquitectónico	25
C.4. Diseño procedimental	26
Apéndice D Documentación técnica de programación	29
D.1. Introducción	29
D.2. Estructura de directorios	29

D.3. Manual del programador	29
D.4. Compilación, instalación y ejecución del proyecto	29
D.5. Pruebas del sistema	29
Apéndice E Documentación de usuario	31
E.1. Introducción	31
E.2. Requisitos de usuarios	31
E.3. Instalación	31
E.4. Manual del usuario	31
Apéndice F Anexo de sostenibilización curricular	33
F.1. Introducción	33
Bibliografía	35

Índice de figuras

A.1. Burndown Report del Sprint 8	11
B.1. Diagrama de Casos de Uso	14
C.1. Mockup de la web	27
C.2. Diagrama de Flujo de la Aplicación	28
C.3. Diagrama de Secuencia de la aplicación	28

Índice de tablas

Apéndice A

Plan de Proyecto Software

A.1. Introducción

La planificación de un proyecto software constituye un proceso de gestión organizado que permite coordinar actividades, recursos y plazos con el fin de garantizar una ejecución controlada y la consecución de los objetivos definidos. En el contexto de este TFG, donde se desarrolla una aplicación web para la inferencia y visualización de sendas de herbívoros a partir de imágenes aéreas, disponer de un plan de proyecto resulta especialmente relevante para estructurar el trabajo y registrar el avance de forma verificable.

Este apartado presenta, en primer lugar, la planificación temporal seguida durante el desarrollo, desglosando el proyecto en tareas e hitos y estableciendo su evolución cronológica. Para ello, se adopta un enfoque iterativo e incremental inspirado en metodologías ágiles, particularmente *Scrum*, organizando el trabajo en ciclos cortos llamados *Sprints* con los que se busca conseguir una correcta revisión del progreso y una posible adaptación ante incidencias o cambios de alcance, manteniendo una visión continua del conjunto de tareas pendientes y completadas mediante el denominado *Product Backlog*, que funciona como un registro de las labores a llevar a cabo.

Finalmente, el plan se completa con un estudio de viabilidad que analiza el proyecto desde una perspectiva económica y legal. En esta parte se consideran los costes asociados al desarrollo y despliegue, así como las implicaciones derivadas del uso de herramientas, librerías y recursos de terceros, prestando especial atención a licencias software y condiciones de

uso, con el objetivo de asegurar que la solución propuesta sea sostenible, reutilizable y coherente con un escenario de explotación real.

A.2. Planificación temporal

Sprint 1 - Documentación e investigación de conceptos teóricos

El *Sprint 1* se desarrolló del 25 de octubre al 7 de noviembre. En esta primera etapa, antes del inicio del segundo semestre, los *sprints* estaban pensados para una duración de dos semanas, y funcionó como una toma de contacto inicial con el TFG, ya que comencé a trabajar en él antes de lo establecido oficialmente: aunque la asignatura la curso en el segundo semestre, empecé a avanzar a finales del primero. Además, la semana siguiente a iniciar el TFG comencé mis prácticas en empresa y, junto con tres asignaturas en paralelo, me resultó complicado reservar horas estables para esta tarea, aunque fui sacando tiempo progresivamente.

Para este *sprint* estimé una dedicación total de 22 horas. Finalmente, dediqué aproximadamente 15 horas, lo que se reflejó en que no llegué a completar todas las *issues* previstas.

Las tareas planteadas para este *sprint* fueron:

- Documentación e investigación de conceptos teóricos asociados al proyecto (segmentación semántica, predicción densa, aprendizaje profundo, evaluación y métricas, *encoders* y modelos/arquitecturas).
- Estudio en detalle del artículo base sobre el que se apoya el TFG, para comprender su fundamentación teórica y el enfoque metodológico.
- Localización y recopilación de bibliografía para los distintos conceptos a documentar.
- Configuración del repositorio y del flujo de trabajo: GitHub, operación remota y entorno de desarrollo en VS Code.
- Puesta en marcha del entorno $\text{\LaTeX}$ con la plantilla proporcionada por los profesores y configuración de herramientas (TeXstudio, MiKTeX Console y TeXworks).
- Selección y configuración de la herramienta de gestión de tareas, optando por Zube.io tras descartar ZenHub.

Durante la primera semana me centré principalmente en asentar el entorno de trabajo. Configuré GitHub y el acceso para operación remota, dejé preparado VS Code para trabajar con comodidad y monté la plantilla de L^AT_EX proporcionada por los profesores. En paralelo, instalé y ajusté las herramientas necesarias para editar y compilar el proyecto, *TeXstudio*, *MiKTeX Console* y *TeXworks*, ya que al principio necesitaba entender bien cómo organizar el proyecto y compilar sin fricción. También exploré opciones para la gestión del tablero de trabajo: tras comprobar que ZenHub me daba un error inesperado y sin mucha posibilidad de solución, migré a Zube.io y dejé preparada la estructura para organizar las *issues*.

A lo largo de las dos semanas, la parte principal del *sprint* fue la investigación y búsqueda bibliográfica. Dediqué la mayor parte del tiempo a leer y estudiar diferentes *papers* científicos para asentar la base teórica, especialmente el artículo principal en el que se fundamenta el TFG, que trabajé en detalle para comprender correctamente su planteamiento. Como resultado de esta fase, conseguí recopilar bibliografía para la mayoría de los conceptos que tenía previstos documentar, aunque la redacción completa no llegó a cerrarse en este *sprint*.

En cuanto a la documentación escrita, el objetivo era redactar las *issues* de conceptos teóricos planificadas; sin embargo, por falta de tiempo y por la incertidumbre inicial sobre cómo estructurar el contenido, únicamente avancé en la redacción de dos bloques: la definición de *Visión por Computador* y los conceptos teóricos generales de la *Segmentación Semántica*. Además, estos avances no quedaron consolidados mediante *commit* hasta el *sprint* siguiente, ya que durante este primer tramo estuve reorganizando la estructura del proyecto y me costó definir correctamente las bases del mismo.

Sprint 2 - Primer prototipo de la aplicación web

El *Sprint 2* abarcó del 8 de noviembre al 22 de noviembre. Para este periodo estimé una dedicación de 12,5 horas, aunque finalmente empleé 21 horas. En este punto todavía no tenía definida una planificación basada en *points* y, por tanto, no pude reflejar correctamente el avance mediante *burndown reports*. En su lugar, intenté organizarme con un campo personalizado de *horas*, pero comprobé que no resultaba útil para medir progreso ni para generar informes de seguimiento.

Los objetivos principales de este *sprint* fueron los siguientes:

- Estudiar la documentación oficial de Flask y realizar los tutoriales propuestos [?].

- Diseñar el *mockup* inicial de la aplicación web.
- Implementar una primera versión sencilla de la web, con cuatro botones principales.
- Verificar el flujo de entrada/salida de la aplicación: recibir una imagen en Flask y dibujar en HTML unas líneas de esquina a esquina sobre la imagen.
- Añadir documentación del código y una primera batería de *tests* asociados a este acercamiento inicial.

Durante la primera semana me centré en dos frentes. Por un lado, diseñé el *mockup* de la web para fijar la estructura de pantallas y el flujo esperado de uso. Por otro, dediqué tiempo a estudiar Flask, siguiendo la documentación oficial y completando los tutoriales, ya que necesitaba asentar la base del *backend* antes de comenzar con una implementación funcional.

En la segunda semana abordé la implementación inicial de la aplicación. Construí un prototipo sencillo con los cuatro botones definidos y dejé operativo el flujo mínimo: la aplicación recibía una imagen en el servidor y se mostraba en el *frontend* con un trazado simple de líneas de esquina a esquina. Este paso fue clave para validar de forma temprana el formato del JSON de trazas y confirmar que el intercambio de datos entre *backend* y *frontend* estaba funcionando como esperaba. Además, documenté el código y añadí *tests* básicos para asegurar que este acercamiento inicial quedaba mínimamente cubierto.

A lo largo del *sprint* también intenté avanzar en la redacción de conceptos teóricos arrastrados del *sprint* anterior, pero apenas conseguí progresar en esa parte debido a la carga de trabajo asociada a la puesta en marcha del prototipo. Aun así, sí logré completar las tareas específicas definidas para este *sprint*. La única excepción fue la integración del *mockup* en la memoria, que no pude dejar lista a tiempo por un fallo en la configuración del entorno de L^AT_EX en TeXstudio, lo que obligó a posponer esa incorporación.

Sprint 3 - Ajustes en estilo y aplicación de la web

El *Sprint 3* se desarrolló entre el 22 de noviembre y el 6 de diciembre. Para este *sprint* estimé inicialmente 4,5 horas, ya que en ese momento todavía no estaba creando todas las tareas de forma consistente y únicamente llegué a registrar algunas. Sin embargo, la dedicación real fue notablemente superior,

alcanzando aproximadamente 18 horas, debido a que se fueron abriendo frentes adicionales de revisión y ajuste conforme avanzaba el prototipo.

Cabe destacar que, durante la reunión que dio inicio al *sprint*, conocí al que sería mi otro tutor del TFG, David Martínez, quien me orientó sobre varios aspectos relacionados con el estilo de la interfaz, como Bootstrap, Tailwind y alternativas, y sobre ciertos requerimientos esperables en la web, lo que condicionó parte de las decisiones técnicas de este periodo.

Los objetivos abordados durante este *sprint* fueron los siguientes:

- Sustituir el botón de “dibujar trazas” por un *checkbox*, simplificando el flujo de interacción.
- Corregir la funcionalidad asociada al PNG, de forma que la web dibujase las trazas sobre el mismo PNG original en lugar de generar un PNG nuevo.
- Investigar Bootstrap y TailwindCSS, y elaborar una comparativa entre ambos enfoques.
- Estudiar y preparar la adopción de DaisyUI como capa de componentes sobre Tailwind.
- Realizar limpieza del código, mejorar su documentación y aplicar pequeños ajustes locales para verificación de versiones.
- Continuar, de forma paralela, con la investigación teórica y la recopilación bibliográfica de conceptos arrastrados del *sprint* 1.

Durante los primeros días me centré en mejorar el prototipo existente y en cerrar cambios funcionales pequeños pero importantes. En primer lugar, sustituí el botón dedicado a dibujar trazas por un *checkbox*, ya que resultaba más coherente con el tipo de acción y evitaba pasos innecesarios en el uso. En paralelo, corregí la funcionalidad asociada al PNG: hasta ese momento, mi aproximación consistía en generar un archivo nuevo con las trazas, pero el comportamiento esperado era que la aplicación dibujase directamente sobre el mismo PNG que se estaba visualizando. Este cambio fue relevante porque simplificó el flujo de representación y alineó la salida con el objetivo de validación visual de las trazas.

Además, en esta primera semana dediqué tiempo a investigar y comparar Bootstrap y TailwindCSS, con el objetivo de entender qué enfoque se ajustaba mejor a una interfaz sencilla pero mantenible. Como resultado, dejé

documentada una comparativa que me permitió aclarar las diferencias entre un *framework* basado en componentes predefinidos (Bootstrap) y uno basado en utilidades (Tailwind), teniendo en cuenta el grado de personalización, la curva de aprendizaje y el impacto en el mantenimiento del proyecto. Estos cambios quedaron reflejados junto con una primera limpieza y documentación del código, de forma que el prototipo quedase más legible de cara a iteraciones posteriores.

En la segunda semana el foco pasó a DaisyUI. Dediqué tiempo a estudiar este conjunto de componentes sobre Tailwind, explorando su filosofía y la forma de integrarlo en un proyecto web, ya que la idea era disponer de componentes consistentes sin perder la flexibilidad de Tailwind. En este proceso también realicé algunos ajustes locales para verificación de versiones y reorganicé parte del proyecto eliminando dependencias que no estaban aportando valor en ese momento, con la intención de reducir complejidad antes de iniciar una migración estética más profunda.

Por último, durante todo el *sprint* mantuve un trabajo paralelo de investigación teórica asociado a conceptos que venía arrastrando del *sprint* 1. Aunque no llegué a consolidar estos avances en *commits* específicos, continué revisando bibliografía y tomando notas sobre bloques pendientes, como, por ejemplo, aspectos relacionados con predicción densa, tipos de arquitecturas y métricas de evaluación, con la intención de retomar su redacción cuando el entorno técnico de la web quedase más estabilizado.

Sprint 7 - Revisión de la memoria y cumplimiento de objetivos pasados

El *Sprint 7* abarcó el periodo comprendido entre el 21 de enero y el 10 de febrero. Se trató de un intervalo especialmente amplio, en el que se concentraron cambios relevantes tanto en la memoria como en la implementación, en consonancia con el cierre de mi periodo de prácticas y el consecuente incremento del tiempo dedicado al TFG. En conjunto, planifiqué una dedicación total de 39,5 horas para este *sprint*, aunque finalmente empleé 54 horas.

Los objetivos definidos para este *sprint* fueron los siguientes:

1. Revisar y reformular el apartado de objetivos del proyecto, con el propósito de mejorar su claridad y consistencia.
2. Refinar explicaciones del Capítulo 3, *Conceptos teóricos*, corrigiendo definiciones puntuales.

3. Reestructurar el Capítulo 6, *Trabajos relacionados*, incorporando mayor apoyo visual y completando explicaciones incompletas.
4. Redactar los bloques conceptuales pendientes correspondientes a:
 - Métricas de evaluación en segmentación semántica,
 - Tipos de *encoder*.
5. Integrar en la aplicación web la lógica del algoritmo de inferencia proporcionado.
6. Adaptar la batería de tests a los cambios introducidos.
7. Generar un fichero `requirements.txt` con las dependencias de Python del proyecto.
8. Elaborar el diagrama de casos de uso y documentar los casos asociados.

En cuanto a la revisión de los objetivos, la organicé en dos días. El primero lo dediqué a identificar las descripciones que resultaban menos precisas y a analizar memorias de años anteriores para ajustar el estilo al esperado, mientras que el segundo se centró en la reescritura y el refinamiento final del texto.

Paralelamente, corregí definiciones concretas del Capítulo 3, en particular las relativas a *Vision Transformer* y a los mecanismos residuales. Asimismo, incorporé un texto introductorio al Capítulo 6, revisé algunas redacciones y ajusté *captions* de las figuras. Finalmente, añadí una tabla comparativa entre trabajos relacionados, cuya construcción y ajuste visual realicé en dos iteraciones para asegurar su correcta visualización en el documento.

Más adelante, completé los conceptos pendientes, correspondientes a métricas de evaluación y tipos de *encoder*. La elaboración se distribuyó en cinco días: dos destinados a métricas de evaluación, primero para redactar el contenido y después para consolidar bibliografía, fórmulas y recursos visuales, y tres destinados a *encoders*, donde fue necesario realizar una revisión bibliográfica más amplia antes de cerrar la redacción.

El hito central de este *sprint* fue la integración en la aplicación web de la lógica de inferencia del *notebook* proporcionado. Tras analizar el flujo completo, abordé una primera aproximación mediante serialización con *pickle*, la cual no conseguí llegar a implementar del todo correctamente. Por ello, opté por una solución más robusta basada en convertir los modelos por *fold* a módulos *TorchScript* autocontenidos, ya normalizados y directamente

integrables en la aplicación. Posteriormente, adapté la lógica del sistema para ejecutar esta inferencia de forma funcional, cerrando esta parte el 1 de febrero.

Como consecuencia de los cambios introducidos, actualicé la batería de tests para alinearla con la nueva integración. Adicionalmente, por recomendación de uno de los tutores, generé el fichero `requirements.txt` con las versiones de las dependencias utilizadas hasta el momento, al disponer ya de la información necesaria.

Por último, finalicé otra tarea arrastrada de *sprints* anteriores: la creación del diagrama de casos de uso y la documentación asociada. Para ello, me apoyé en los *mockups* y en el flujo de uso previsto, lo que facilitó mantener una estructura coherente y cerrada.

Cabe destacar que, antes de la reunión que dio inicio al siguiente *sprint*, se realizó una reunión intermedia de seguimiento. Sin embargo, al considerar que todavía quedaban varios frentes abiertos y que el avance no era suficientemente cerrado, se decidió prolongar el *sprint* una semana adicional para consolidar entregables y reducir la deuda técnica acumulada.

Sprint 8 - Últimos pasos antes de la release

Este *sprint* se desarrolló del 10 al 18 de febrero, finalizando todas las tareas un día antes de lo previsto, ya que, inicialmente se contemplaba el 19 como fecha de fin. El objetivo principal era completar las últimas labores previas a la primera *release* del proyecto. No obstante, durante el avance surgieron varias dudas y fallos que condicionaban el cierre, principalmente porque el modelo que se estaba utilizando para calcular los datos de las trazas no era el adecuado, lo que hizo necesario planificar un *sprint* adicional para consolidar esta parte.

En conjunto, estimé una dedicación total de 27 horas para este *sprint*, y finalmente empleé 28 horas y 35 minutos.

Las tareas abordadas fueron las siguientes:

- Comparativa del modelo basado en *pickle* con el de PyTorch.
- Modificación de la lógica de *upload* y *calculate* de la aplicación.
- Creación de una *cookie* de sesión con caducidad a las 24 h.
- Adaptación de la aplicación a diseño *responsive*.

- Creación de la infraestructura base para integrar Flask-Babel.
- Revisión de varios puntos de la memoria.
- Actualización del diagrama de casos de uso.
- Sustitución del CSS por DaisyUI.

En cuanto a las labores de documentación, revisé varios puntos de la memoria: añadí el nombre del alumno, el de los tutores y el título del TFG, redacté un párrafo introductorio para el Capítulo 3: *Conceptos teóricos*, corregí detalles del Capítulo 6: *Trabajos relacionados* relacionados con imágenes y redacción, y ajusté el apartado C de anexos correspondiente al diseño. En este último caso, reorganicé contenido ya que gran parte de los puntos estaban realmente vinculados al apartado 4 de la memoria: *Técnicas y Herramientas*. Aunque se trataba de muchas tareas, eran intervenciones cortas y muy localizadas.

También, dentro de los anexos, ajusté el diagrama de casos de uso para mejorar su interpretabilidad desde un punto de vista externo. En concreto, reordené la aparición de los actores, modifiqué algún caso de uso, añadí nuevas tablas para los casos incorporados y completé el campo *exception* en todas ellas para mantener la consistencia del formato.

Por otro lado, en la aplicación se llevaron a cabo varias mejoras. En primer lugar, modifiqué la lógica de las funciones de *upload* y *calculate*. Antes, al insertar una imagen, esta se subía directamente al *backend*, lo que podía provocar subidas innecesarias si el usuario seleccionaba un archivo incorrecto. Para evitarlo, implementé un flujo en el que la imagen se carga en local y solo se envía al servidor cuando el usuario pulsa *calcular trazas*, momento en el que ya ha podido verificarla. Para ello, creé una función nueva que orquestase las dos operaciones ya existentes.

A continuación, establecí un límite de sesión de 24 h mediante *cookies*, de forma que el usuario pudiera mantener su progreso durante un margen razonable. En paralelo, trabajé en el diseño *responsive* para que, en pantallas más pequeñas, como dispositivos móviles o *tablets*, se visualizasen correctamente todos los elementos y la interfaz resultase usable en distintos dispositivos.

Más adelante, preparé la infraestructura de Flask-Babel en dos días: el primero lo dediqué a revisar la documentación y a entender su funcionamiento, y el segundo a implementar los elementos base. En esta fase instalé la biblioteca y la incorporé al `requirements.txt`, creé el archivo `Babel.cfg`

para la extracción de textos, añadí un desplegable con cinco idiomas: español, inglés, francés, italiano y alemán, generé recursos visuales en formato `.svg` para las banderas asociadas y dejé definidos algunos campos de Babel, sin llegar todavía a implementar la lógica completa de traducción.

La tarea principal del *sprint* fue migrar toda la interfaz de la aplicación a DaisyUI sobre Tailwind. Este proceso lo realicé de forma progresiva durante todos los días del *sprint*, planteándome pequeños avances diarios hasta completar la migración. Primero investigué los componentes disponibles y los fui aplicando iterativamente: comencé por los botones principales: insertar imagen, calcular trazas y borrar imagen, después ajusté los modales y el sistema de *flash messaging* aprovechando los diseños predefinidos, estableciendo además que los avisos se mostrasen durante cinco segundos antes de desaparecer. Más tarde, reorganicé la zona de visualización de la imagen e incorporé una lógica para permitir el arrastre de archivos desde una carpeta externa, ya que resultaba conveniente para el flujo de uso. Durante este proceso también corregí la cabecera, añadí un *footer* con información del proyecto y del ente académico correspondiente, e incorporé una imagen de fondo suave para aportar una estética más cálida.

Finalmente, el último día realicé la comparativa entre el modelo proporcionado mediante *pickle* y el modelo en PyTorch que yo había generado. Comparé métricas de rendimiento y de salida, incluyendo tiempos de carga desde disco, tiempo medio de inferencia y desviación estándar, *IoU* y *Dice* entre predicciones, número de píxeles clasificados como positivos, longitud del esqueleto en píxeles, cantidad de componentes conexas y grosor medio estimado, entre otros. Tras el análisis, concluí que el modelo en PyTorch perdía demasiada capacidad lógica frente al margen de mejora temporal obtenido, por lo que resultaba conveniente dedicar un *sprint* adicional antes del cierre de la *release* para dejar esta lógica correctamente integrada.

Como se observa en el *burndown* A.1, la reducción de *open points* fue más lenta que la ideal durante la mayor parte del *sprint*, debido a los ajustes iterativos en la interfaz y la infraestructura, tareas que ocupaban un gran número de puntos, aunque fueron siendo ejecutadas durante toda la semana; no obstante, en los últimos días se cerró un bloque grande de trabajo, quedando el *sprint* prácticamente completado el 18 de febrero, un día antes de lo planificado.

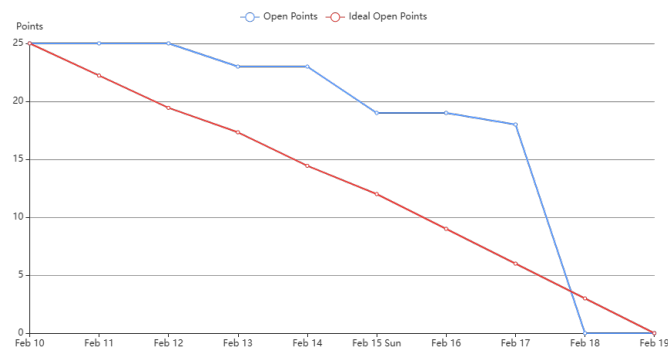


Figura A.1: Burndown Report del Sprint 8.

A.3. Estudio de viabilidad

Viabilidad económica

Viabilidad legal

Apéndice *B*

Especificación de Requisitos

B.1. Introducción

B.2. Objetivos generales

B.3. Catálogo de requisitos

B.4. Especificación de requisitos

En esta sección se presenta la especificación funcional de la aplicación mediante casos de uso, describiendo de forma estructurada las interacciones que el usuario puede realizar a través de la interfaz. Dado que la aplicación se consume desde el cliente web, el diagrama sintetiza las funcionalidades visibles para cada tipo de actor, mientras que el *backend* se considera un soporte imprescindible para ejecutar la inferencia, gestionar el estado de las imágenes y servir los resultados, sin modelarse explícitamente como actor independiente.

Diagrama de Casos de Uso

En la figura [B.1](#) se recoge el diagrama general de casos de uso y, a continuación, se detallan las descripciones individuales de cada caso representado. Estas tablas se han derivado a partir del flujo de uso previsto y de los *mocups* de la interfaz, con el objetivo de concretar precondiciones, secuencias principales, relaciones entre funcionalidades y el nivel de importancia de cada operación dentro del *pipeline* de la aplicación.

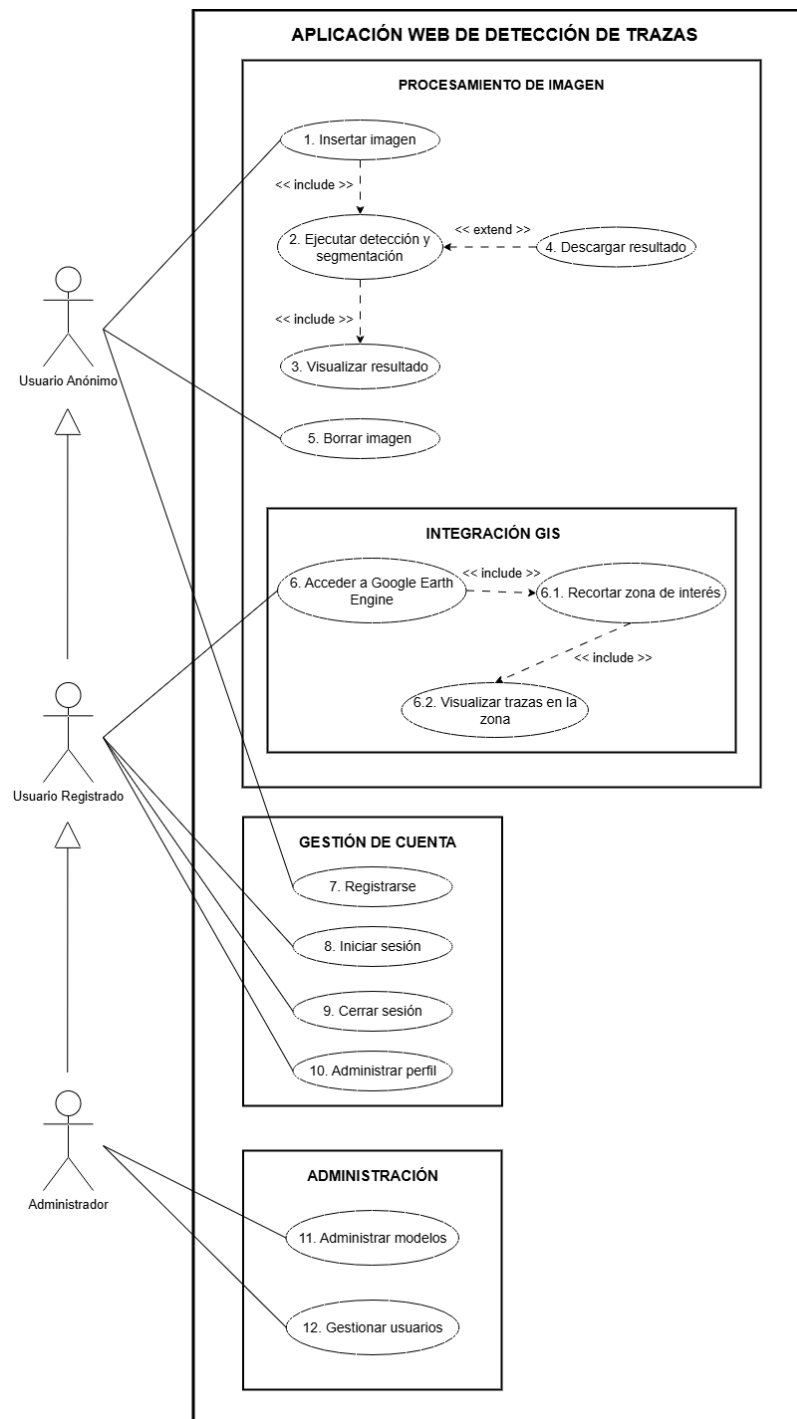


Figura B.1: Diagrama de casos de uso de la aplicación

B.5. Actores del sistema

En esta sección se describen los distintos actores que interactúan con la aplicación web de detección de trazas. Los actores representan los perfiles de usuario y roles funcionales que acceden al sistema, diferenciándose en función de sus permisos y capacidades dentro de la aplicación. Esta distinción permite modelar de forma clara las responsabilidades y el alcance de cada tipo de usuario.

Usuario Anónimo

El **Usuario Anónimo** representa a cualquier persona que accede a la aplicación sin haber iniciado sesión ni disponer de una cuenta registrada. Este actor tiene acceso a las funcionalidades básicas del sistema, orientadas a la demostración y uso general de la herramienta de detección.

Concretamente, el Usuario Anónimo puede:

- Cargar una imagen para su análisis.
- Ejecutar el proceso de detección y segmentación de trazas.
- Visualizar el resultado obtenido.
- Borrar la imagen cargada y reiniciar el flujo de procesamiento.
- Registrarse en el sistema.

Este perfil permite el uso inmediato de la aplicación sin barreras de acceso, manteniendo restringidas aquellas funcionalidades que requieren persistencia de datos o integración con servicios externos.

Usuario Registrado

El **Usuario Registrado** es un agente que dispone de una cuenta en el sistema y ha iniciado sesión correctamente. Este actor hereda todas las capacidades del Usuario Anónimo y, adicionalmente, puede acceder a funcionalidades avanzadas que requieren autenticación.

Además de las acciones disponibles para el Usuario Anónimo, el Usuario Registrado puede:

- Iniciar y cerrar sesión en el sistema.

- Administrar su perfil de usuario.
- Descargar los resultados obtenidos tras el proceso de detección y segmentación.
- Acceder al módulo de integración GIS mediante Google Earth Engine.
- Definir una zona de interés y visualizar trazas dentro de dicha zona.

Este actor representa el perfil principal de uso de la aplicación, proporcionando acceso a funcionalidades de mayor valor añadido y a la integración con servicios externos.

Administrador

El **Administrador** es un usuario registrado con privilegios adicionales orientados a la gestión y mantenimiento del sistema. Este actor hereda todas las capacidades del Usuario Registrado y dispone de permisos específicos para la configuración interna de la aplicación.

Las funciones exclusivas del Administrador incluyen:

- Administrar los modelos de detección utilizados por el sistema.
- Seleccionar el modelo activo que se empleará en el proceso de detección y segmentación.

La existencia de este rol permite separar claramente las tareas de uso del sistema de aquellas relacionadas con su configuración y control, facilitando la mantenibilidad y escalabilidad de la aplicación.

Explicación de los casos de uso

CU-01 Insertar imagen (previsualizar)	
Actores	Usuario anónimo, Usuario registrado, Administrador
Descripción	Permite al usuario seleccionar una imagen desde su dispositivo y previsualizarla en el cliente. La imagen no se envía al backend en este paso.
Precondición	El usuario ha accedido a la interfaz de procesamiento.
Flujo principal	1) El usuario pulsa <i>Insertar imagen</i> o la arrastra y selecciona un fichero válido. 2) El sistema carga la imagen en el cliente y la muestra en pantalla como previsualización. 3) El sistema actualiza el estado indicando que puede ejecutarse el cálculo de trazas.
Postcondición	Imagen seleccionada y visible en la interfaz.
Excepciones	E1) El usuario cancela la selección del fichero → no se modifica el estado de la interfaz. E2) El fichero no es un formato permitido → no se carga la previsualización y se informa al usuario. E3) Error al leer el fichero → se notifica el error y no se actualiza la imagen mostrada.
Relaciones	Incluye a CU-02 Ejecutar detección y segmentación.
Importancia	Alta

CU-02 Ejecutar detección y segmentación

Actores	Usuario anónimo, Usuario registrado, Administrador
Descripción	Ejecuta el cálculo de trazas mediante el modelo de detección y segmentación. Si la imagen ha sido seleccionada por el cliente, se envía al backend durante esta operación.
Precondición	Existe una imagen seleccionada en el cliente (CU-01) o una imagen previamente almacenada en el servidor en sesión.
Flujo principal	1) El usuario pulsa <i>Calcular trazas</i> . 2) El sistema envía al backend la imagen seleccionada si procede; en caso contrario, reutiliza la imagen existente en servidor. 3) El backend valida el formato, almacena la imagen y ejecuta la inferencia. 4) El sistema genera y persiste el JSON de trazas asociado a la imagen. 5) El sistema deja disponible la visualización del resultado (CU-03).
Postcondición	Trazas calculadas y disponibles para su visualización.
Excepciones	E1) No existe imagen seleccionada en el cliente ni imagen almacenada en el servidor → se informa al usuario y no se ejecuta el cálculo. E2) Formato no permitido → se rechaza la operación y se informa al usuario. E3) Error durante la inferencia o la segmentación → se muestra un mensaje de error y no se generan trazas. E4) La carga excede el tamaño máximo permitido → se aborta la petición y se notifica el error.
Relaciones	Incluye CU-03 Visualizar resultado.
Importancia	Alta

CU-03 Visualizar resultado

Actores	Usuario anónimo, Usuario registrado, Administrador
Descripción	Permite visualizar el resultado de la detección y segmentación superpuesto sobre la imagen original.
Precondición	El proceso de detección se ha ejecutado correctamente (CU-02).
Flujo principal	1) El sistema solicita el JSON de trazas al backend. 2) El sistema renderiza la imagen y superpone las trazas sobre un canvas. 3) El usuario visualiza el resultado por pantalla.
Postcondición	Resultado visualizado en pantalla.
Excepciones	E1) No existen trazas calculadas → se informa al usuario y no se dibujan trazas. E2) El archivo de trazas no se encuentra o está corrupto → se muestra error y se solicita recalcular. E3) Error de red o de timeout en la obtención del JSON → se notifica y se permite reintentar.
Relaciones	Incluido por CU-02 Ejecutar detección y segmentación.
Importancia	Alta

CU-04 Descargar resultado

Actores	Usuario registrado, Administrador
Descripción	Permite descargar el resultado de la detección y segmentación para su uso externo.
Precondición	El resultado ha sido generado y visualizado (CU-03).
Flujo principal	1) El usuario solicita la descarga del resultado. 2) El sistema genera el archivo correspondiente. 3) El archivo es descargado por el usuario.
Postcondición	Resultado almacenado localmente por el usuario.
Excepciones	E1) No existe resultado disponible → se informa y se cancela la descarga. E2) Usuario no autorizado o no autenticado → se deniega el acceso. E3) Error generando el recurso descargable → se notifica y se permite reintentar.
Relaciones	Extiende CU-02 Ejecutar detección y segmentación.
Importancia	Media-Alta

CU-05 Borrar imagen

Actores	Usuario anónimo, Usuario registrado, Administrador
Descripción	Permite eliminar la imagen cargada y reiniciar el flujo de procesamiento.
Precondición	Existe una imagen una imagen en previsualización local o un resultado generado.
Flujo principal	1) El usuario solicita borrar la imagen. 2) Si solo existe previsualización local, el sistema limpia la imagen y el canvas en el cliente. 3) Si existe imagen en servidor, el sistema solicita al backend la eliminación de la imagen y sus trazas asociadas. 4) El sistema vuelve al estado inicial.
Postcondición	Sistema preparado para una nueva ejecución.
Excepciones	E1) No existe ninguna imagen para borrar → se informa al usuario. E2) Error eliminando recursos en servidor → se notifica y se mantiene el estado consistente. E3) Error de comunicación con el backend → se informa y se permite reintentar.
Importancia	Media

CU-06 Acceder a Google Earth Engine

Actores	Usuario registrado, Administrador
Descripción	Permite acceder al módulo de integración con Google Earth Engine para trabajar sobre una zona geográfica.
Precondición	El usuario ha iniciado sesión.
Flujo principal	1) El usuario accede al módulo GIS. 2) El sistema establece conexión con Google Earth Engine. 3) Se habilitan las funcionalidades de recorte y visualización.
Postcondición	Sesión GIS activa.
Excepciones	E1) Error de autenticación o de autorización con el servicio → se deniega el acceso y se informa al usuario. E2) Servicio no disponible o error de red → se notifica indisponibilidad temporal. E3) Configuración incompleta → se informa y se bloquea el acceso al módulo.
Relaciones	Incluye CU-06.1 Recortar zona de interés y CU-06.2 Visualizar trazas en la zona.
Importancia	Media-Alta

CU-06.1 Recortar zona de interés

Actores	Usuario registrado, Administrador
Descripción	Permite definir una zona geográfica de interés mediante un recorte sobre el visor.
Precondición	Acceso al módulo GEE (CU-06).
Flujo principal	1) El usuario define el área de recorte. 2) El sistema valida la geometría. 3) La zona queda establecida como área activa.
Excepciones	E1) Geometría inválida → se rechaza el recorte y se solicita corrección. E2) Zona fuera de límites permitidos → se informa y se requiere ajuste. E3) Error comunicándose con el servicio → se notifica y se permite reintentar.
Postcondición	Zona de interés definida.
Importancia	Media

CU-06.2 Visualizar trazas en la zona

Actores	Usuario registrado, Administrador
Descripción	Permite visualizar las trazas detectadas dentro de la zona de interés seleccionada.
Precondición	Zona de interés definida (CU-06.1).
Flujo principal	1) El sistema consulta las trazas dentro de la zona. 2) El sistema renderiza las trazas sobre el visor. 3) El usuario inspecciona el resultado.
Postcondición	Trazas visualizadas en la zona seleccionada.
Excepciones	E1) No existen trazas en la zona → se informa al usuario y se muestra la zona sin trazas. E2) Error de consulta o de renderizado → se notifica y se permite reintentar. E3) Zona no definida → se solicita definir recorte (CU-06.1).
Importancia	Media-Alta

CU-07 Registrarse

Actores	Usuario anónimo
Descripción	Permite crear una cuenta de usuario para acceder a funcionalidades restringidas del sistema.
Precondición	El usuario no tiene sesión iniciada.
Flujo principal	1) El usuario accede al formulario de registro. 2) El usuario introduce los datos requeridos. 3) El sistema valida y crea la cuenta.
Postcondición	Cuenta registrada y disponible para iniciar sesión.
Excepciones	E1) Datos inválidos o incompletos → se informa y no se crea la cuenta. E2) Usuario o correo ya existentes → se rechaza el registro. E3) Error interno de persistencia → se notifica y se permite reintentar.
Importancia	Media

CU-08 Iniciar sesión

Actores	Usuario registrado, Administrador
Descripción	Permite autenticar al usuario y habilitar las funcionalidades según su rol.
Precondición	El usuario dispone de credenciales válidas.
Flujo principal	1) El usuario introduce credenciales. 2) El sistema valida las credenciales. 3) El sistema inicia sesión y aplica permisos.
Postcondición	Sesión iniciada con permisos asociados al rol.
Excepciones	E1) Credenciales incorrectas → se informa y no se inicia sesión. E2) Cuenta deshabilitada o no verificada → se deniega el acceso. E3) Error interno de autenticación → se notifica y se permite reintentar.
Importancia	Media

CU-09 Cerrar sesión

Actores	Usuario registrado, Administrador
Descripción	Finaliza la sesión activa del usuario.
Precondición	Sesión iniciada previamente.
Flujo principal	1) El usuario solicita cerrar sesión. 2) El sistema invalida la sesión y revoca permisos. 3) El sistema redirige al estado de usuario anónimo.
Postcondición	Sesión finalizada.
Excepciones	E1) Sesión ya expirada → el sistema fuerza estado anónimo sin error. E2) Error interno al cerrar sesión → se notifica y se fuerza cierre.
Importancia	Baja-Media

CU-10 Administrar perfil

Actores	Usuario registrado, Administrador
Descripción	Permite consultar y actualizar la información básica del perfil del usuario.
Precondición	Sesión iniciada.
Flujo principal	1) El usuario accede a su perfil. 2) El sistema muestra la información disponible. 3) El usuario modifica los campos permitidos. 4) El sistema valida y guarda los cambios.
Postcondición	Perfil actualizado, si procede.
Excepciones	E1) Datos inválidos → se rechazan cambios y se informa. E2) Error de persistencia → se notifica y se mantiene el perfil anterior. E3) Sesión no válida → se solicita iniciar sesión.
Importancia	Baja-Media

CU-11 Administrar modelos

Actores	Administrador
Descripción	Permite gestionar la configuración del modelo de detección y segmentación utilizada por el sistema.
Precondición	Sesión iniciada como administrador.
Flujo principal	1) El administrador accede al módulo de administración. 2) El sistema muestra las opciones disponibles de configuración de modelos. 3) El administrador selecciona el modelo activo. 4) El sistema guarda la configuración.
Postcondición	Configuración de modelo actualizada.
Excepciones	E1) Selección inválida → se rechaza el cambio y se informa. E2) Modelo o pesos no disponibles → se notifica y se mantiene la configuración previa. E3) Error guardando la configuración → se informa y no se aplican cambios.
Importancia	Media-Alta

CU-12 Gestionar usuarios	
Actores	Administrador
Descripción	Permite al administrador realizar tareas de administración sobre las cuentas de usuario del sistema, incluyendo consulta del listado, modificación de atributos básicos y gestión del estado de las cuentas.
Precondición	El actor ha iniciado sesión y dispone de permisos de administrador.
Flujo principal	1) El administrador accede a la sección de gestión de usuarios. 2) El sistema muestra el listado de usuarios registrados y su información básica. 3) El administrador selecciona un usuario sobre el que operar. 4) El administrador ejecuta una operación de gestión. 5) El sistema valida los cambios y los persiste. 6) El sistema confirma la operación y actualiza el listado.
Postcondición	Información y/o estado del usuario actualizado según la operación realizada.
Excepciones	E1) Usuario objetivo inexistente o no accesible → se informa al administrador y se cancela la operación. E2) Operación no permitida → se rechaza el cambio y se notifica. E3) Datos inválidos en la modificación → se rechazan cambios y se solicita corrección. E4) Error interno de persistencia o comunicación con la base de datos → se notifica el error y no se aplican cambios.
Importancia	Media-Alta

Apéndice C

Especificación de diseño

C.1. Introducción

C.2. Diseño de datos

C.3. Diseño arquitectónico

Arquitectura general de la aplicación

La Figura C.2 representa la arquitectura general de la aplicación y las interacciones entre sus distintos componentes, diferenciando claramente el lado cliente y el lado servidor.

El usuario interactúa con la aplicación a través del navegador web, que actúa como intermediario entre la interfaz gráfica y el backend. En el cliente se distinguen dos responsabilidades bien separadas. Por un lado, el HTML renderizado mediante Jinja se encarga de mostrar la estructura de la interfaz, incluyendo botones, imágenes y modales informativos. Por otro lado, la lógica implementada en JavaScript, apoyada en un elemento *canvas*, gestiona la descarga de los resultados en formato JSON y su representación gráfica, sin delegar operaciones de modificación visual al servidor.

El servidor, implementado con Flask, centraliza la lógica de negocio y expone los distintos endpoints a través de un *Blueprint* dedicado a la gestión de trazas. Para mantener el estado de la aplicación entre peticiones HTTP, se utiliza una *session cookie* que almacena información relativa a la imagen activa y a la disponibilidad de resultados calculados. Asimismo, el sistema emplea almacenamiento en disco, dentro del directorio *instance*,

tanto para persistir las imágenes subidas por el usuario como para guardar los resultados intermedios en formato JSON.

Este diseño garantiza una separación clara de responsabilidades, evitando que el cliente modifique directamente recursos en el servidor y relegando el procesamiento visual al navegador, lo que mejora la escalabilidad y simplifica la arquitectura.

Flujo temporal de interacción entre componentes

La Figura C.3 describe de forma cronológica el flujo de interacción entre el usuario, el navegador, el servidor Flask y el sistema de almacenamiento, permitiendo comprender el comportamiento dinámico de la aplicación.

El proceso se inicia cuando el usuario selecciona una imagen desde la interfaz. El navegador envía dicha imagen al servidor mediante una petición *POST upload*, tras lo cual Flask la persiste en disco y actualiza el estado de la sesión. A continuación, el servidor responde con una redirección que provoca la recarga de la página principal, mostrando la imagen cargada.

Posteriormente, cuando el usuario solicita el cálculo de trazas, el servidor ejecuta la lógica correspondiente, genera los resultados en formato JSON y los almacena en el sistema de ficheros, actualizando de nuevo la sesión para reflejar el nuevo estado. La respuesta se materializa en una recarga de la interfaz que incluye un modal de confirmación.

Una vez cerrado el modal, el navegador solicita explícitamente los resultados mediante una petición *GET traces*. El servidor lee el fichero JSON previamente generado y lo devuelve al cliente, donde el código JavaScript se encarga de dibujar las trazas sobre el *canvas* y reflejar visualmente el estado de la aplicación.

Finalmente, si el usuario decide eliminar la imagen, se envía una petición *POST delete*. El servidor elimina tanto la imagen como los resultados asociados, limpia la información de la sesión y devuelve la aplicación a su estado inicial mediante una nueva recarga de la página.

C.4. Diseño procedimental

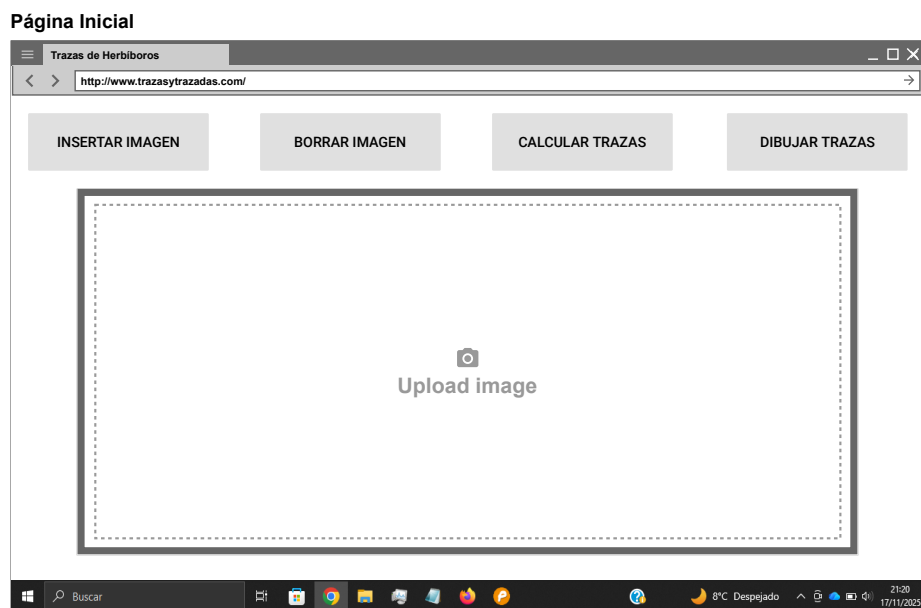


Figura C.1: Mockup de la web

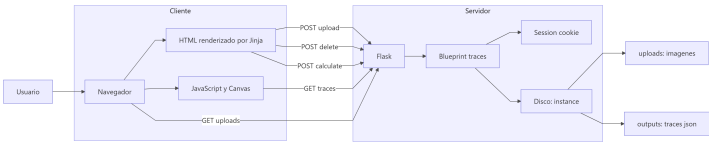


Figura C.2:

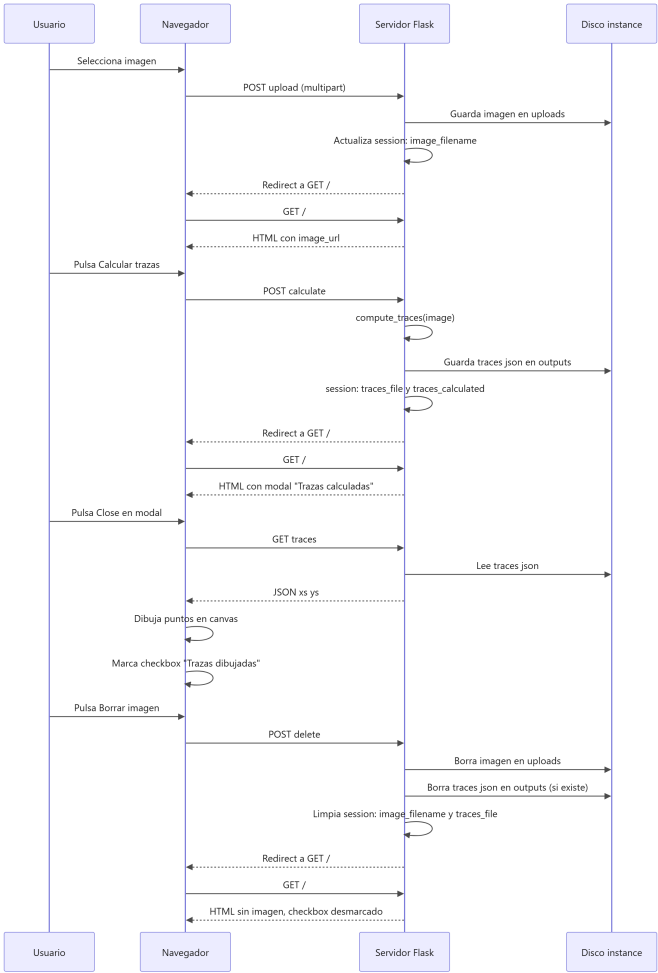


Figura C.3:

Apéndice D

Documentación técnica de programación

- D.1. Introducción
- D.2. Estructura de directorios
- D.3. Manual del programador
- D.4. Compilación, instalación y ejecución del proyecto
- D.5. Pruebas del sistema

Apéndice E

Documentación de usuario

- E.1. Introducción
- E.2. Requisitos de usuarios
- E.3. Instalación
- E.4. Manual del usuario

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo incluirá una reflexión personal del alumnado sobre los aspectos de la sostenibilidad que se abordan en el trabajo. Se pueden incluir tantas subsecciones como sean necesarias con la intención de explicar las competencias de sostenibilidad adquiridas durante el alumnado y aplicadas al Trabajo de Fin de Grado.

Más información en el documento de la CRUE https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf.

Este anexo tendrá una extensión comprendida entre 600 y 800 palabras.

Bibliografía
